Operating System Essentials

Introduction to computer programming

Management and Artificial Intelligence

LUISS

Covering the essentials

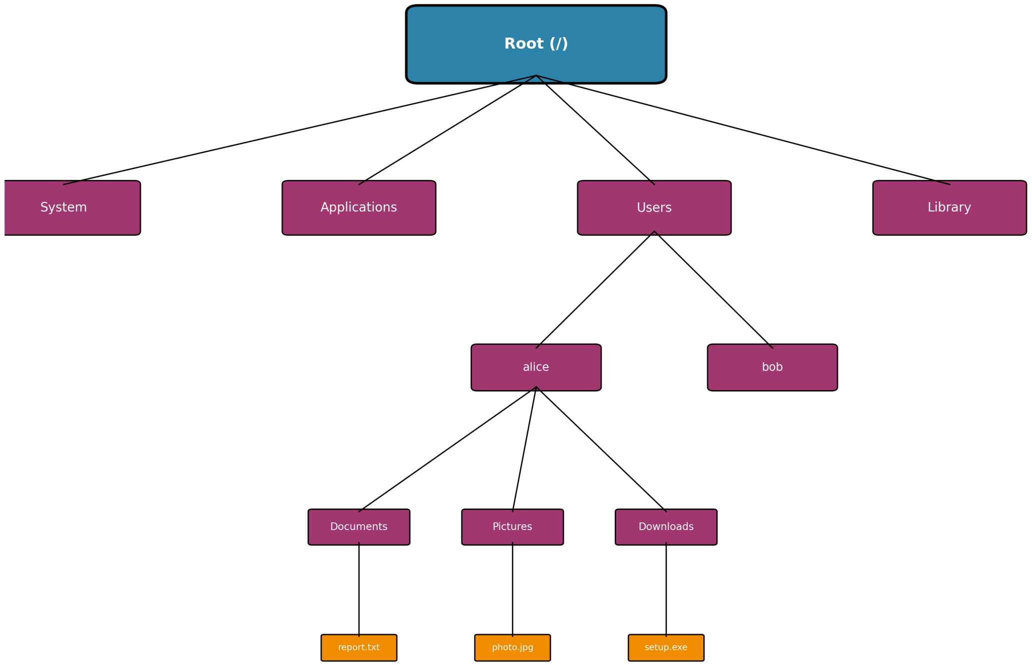
Understanding your operating system is crucial for effective programming and computer use:

- **File Systems**: How files and directories are organized
- **Command Line Interface**: Powerful text-based interaction
- **Process Management**: How programs run and are controlled

We'll focus on **Windows** and **macOS** with hands-on examples!

File Systems: The Foundation

Every operating system organizes files in a **hierarchical structure**, like a tree:



Key Concepts:

- **Root**: The top-level directory (starting point)
- **Directories/Folders**: Containers that hold files and other directories
- **Files**: Individual documents, programs, or data
- **Path**: The route to reach a specific file or directory

Windows File System Structure

Windows uses **drive letters** (C:, D:, etc.) for the root top-level directory and **backslashes** (`\`) to separate folders along the path:

```
C:\                              (Root of C: drive)
├── Windows\                     (System files)
├── Program Files\               (Installed applications)
├── Users\                       (User home directories)
│   ├── Alice\                   (Alice's home directory)
│   │   ├── Desktop\             (Desktop files)
│   │   ├── Documents\           (User documents)
│   │   ├── Downloads\           (Downloaded files)
│   │   └── Pictures\            (User photos)
│   └── Bob\                     (Bob's home directory)
└── temp\                        (Temporary files)
```

Examples of Windows paths:

- `C:\Users\YourName\Documents\report.docx`
- `C:\Program Files\Google\Chrome\chrome.exe`
- `D:\Photos\vacation\beach.jpg`

NOTE: Each disk gets a different drive letter (e.g., **D:**) and an independent file system. By convention, **C:** is assigned to the disk containing the operating system.

macOS File System Structure

macOS uses **forward slashes** (`/`) and starts from root (`/`):

```
/                            (Root directory)
├── Applications/            (Installed applications)
├── System/                  (System files)
├── Users/                   (User home directories)
│   ├── alice/               (Alice's home directory)
│   │   ├── Desktop/         (Desktop files)
│   │   ├── Documents/       (User documents)
│   │   ├── Downloads/       (Downloaded files)
│   │   └── Pictures/        (User photos)
│   └── bob/                 (Bob's home directory)
├── Library/                 (System libraries)
└── tmp/                     (Temporary files)
```

Examples of macOS paths:

- `/Users/YourName/Documents/report.docx`
- `/Applications/Google Chrome.app`
- `/Users/YourName/Pictures/vacation/beach.jpg`

NOTE: macOS mounts other drives within the main filesystem hierarchy. When you connect an external drive like a USB flash drive, it's automatically mounted at `/Volumes/DRIVE_NAME`. For example, if you plug in a flash drive named "PENDRIVE", it would be accessible at: `/Volumes/PENDRIVE/`.

Absolute vs Relative Paths

Absolute Path: Complete path from the root

- Windows: `C:\Users\Alice\Documents\project\main.py`
- macOS: `/Users/alice/Documents/project/main.py`

Relative Path: Path relative to current location

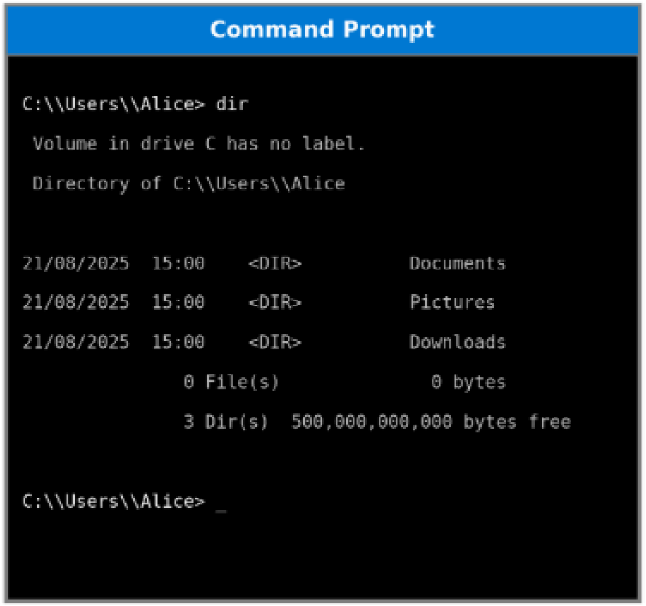
- macOS: `project/main.py` (from current dir, open `project`, reach `main.py`)
- Windows: `project\main.py` (from current dir, open `project`, reach `main.py`)
- macOS: `../Documents/report.docx` (go up one level, then to Documents)
- Windows: `..\Documents\report.docx` (go up one level, then to Documents)
- macOS: `./main.py` (current directory)
- Windows: `.\main.py` (current directory)

Special Directory Symbols:

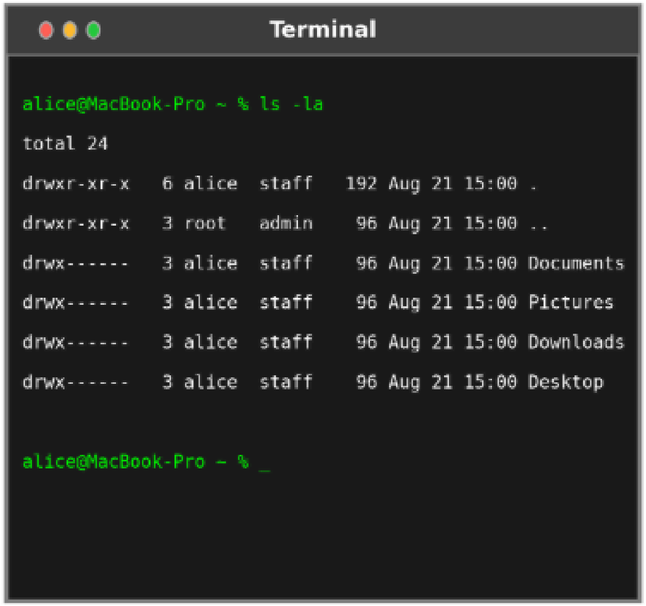
- `.` = current directory
- `..` = parent directory (one level up)
- `~` = home directory (macOS/Linux)
- `%USERPROFILE%` = home directory (Windows)

The Command Line Interface (CLI)

The **command line** is a text-based way to interact with your computer:



Windows Command Prompt



macOS Terminal

Why use the command line?

- **Speed**: Much faster for many tasks
- **Power**: Can do things GUIs can't
- **Automation**: Easy to script repetitive tasks
- **Remote access**: Work on computers anywhere
- **Programming**: Essential for development work

Windows Command Line

Opening the Command Line (alternatives):

- Press `Win + R`, type `cmd`, press Enter (Command Prompt)
- Press `Win + X`, select "Windows PowerShell" (PowerShell)
- Search for "Command Prompt" or "PowerShell" in Start menu

Basic Windows Commands:

Command	Purpose	Example
`dir`	List files and directories	`dir`
`cd`	Change directory	`cd Documents`
`mkdir`	Create directory	`mkdir MyProject`
`copy`	Copy files	`copy file.txt backup.txt`
`del`	Delete files	`del oldfile.txt`
`type`	Display file content	`type readme.txt`
`cls`	Clear screen	`cls`

Practical Exercise: Navigation (Windows Users)

```
# See where you are
cd

# List files in current directory
dir

# Go to your Documents folder
cd Documents

# Create a new folder
mkdir ProgrammingClass

# Enter the new folder
cd ProgrammingClass

# Go back to parent directory
cd ..

# Start a program
notepad.exe
```

macOS Command Line

Opening Terminal (alternatives):

- Press `Cmd + Space`, type "Terminal", press Enter
- Go to Applications → Utilities → Terminal
- Use Spotlight search for "Terminal"

Basic macOS/Unix Commands:

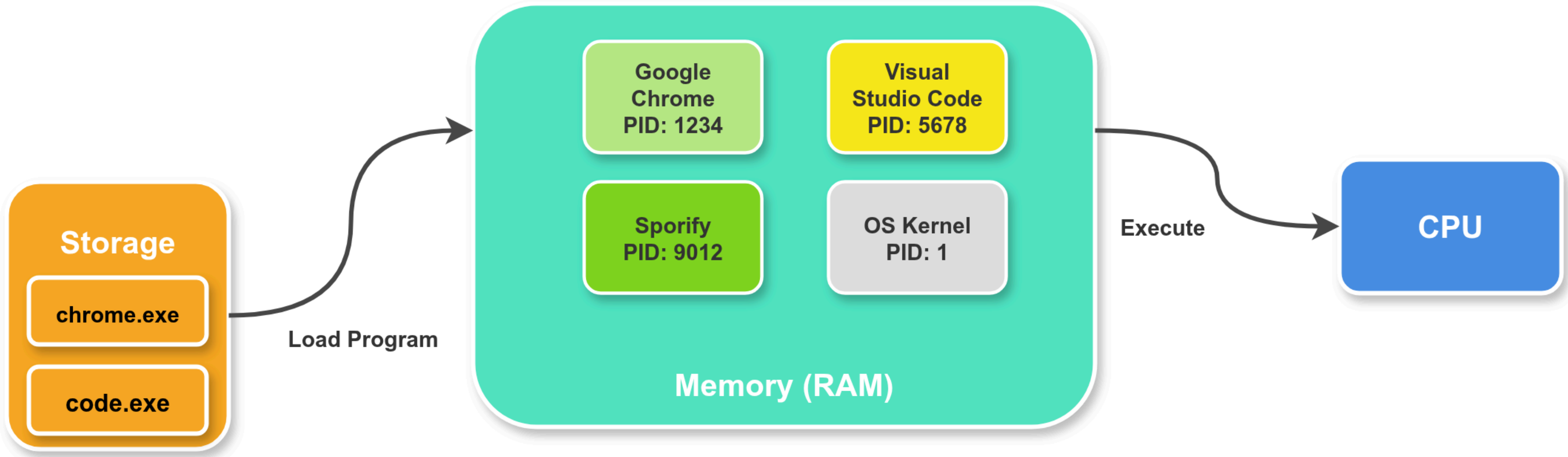
Command	**Purpose**	**Example**
`ls`	List files and directories	`ls -la`
`cd`	Change directory	`cd Documents`
`mkdir`	Create directory	`mkdir MyProject`
`cp`	Copy files	`cp file.txt backup.txt`
`rm`	Delete files	`rm oldfile.txt`
`cat`	Display file content	`cat readme.txt`
`clear`	Clear screen	`clear`

Practical Exercise: Navigation (MacOS Users)

```
# See where you are
pwd

# List files in current directory
ls

# Go to your Documents folder
cd Documents

# Create a new folder
mkdir ProgrammingClass

# Enter the new folder
cd ProgrammingClass

# Go back to parent directory
cd ..

# Start a program
open -a TextEdit
```

Understanding Processes

A **process** is a running program in memory:



Key Process Concepts:

- **Process ID (PID)**: Unique identifier for each process
- **Parent/Child**: Processes can create other processes
- **Memory**: Each process has its own memory space
- **CPU Time**: Processes share CPU time
- **States**: Processes transition between different states:
  - **Running**: Currently executing on the CPU
  - **Ready**: Waiting to be assigned to the CPU
  - **Blocked/Waiting**: Waiting for some event (I/O, signals, etc.)
  - **Terminated**: Execution completed, but still in memory
  - **New**: Process created but not yet admitted to the pool of executable processes

Process Management in Windows

Task Manager (Ctrl + Shift + Esc):

- Graphical interface to view and manage running processes
- Monitor CPU, memory, disk, and network usage
- Terminate unresponsive applications

Command Line Process Management:

```
# List all running processes
tasklist

# Find specific processes by name
tasklist | findstr "notepad"

# Kill a process by name (force with /F)
taskkill /IM notepad.exe
taskkill /F /IM notepad.exe

# Kill a process by PID (force with /F)
taskkill /PID 1234
taskkill /F /PID 1234

# Start a program
start notepad.exe
```

Process Management in macOS

Activity Monitor (Applications → Utilities):

- GUI tool for monitoring and managing processes
- Shows CPU, memory, energy, disk, and network usage

Command Line Process Management:

```
# List running processes
ps aux

# Find specific process
ps aux | grep "TextEdit"

# Kill a process by PID
kill 1234

# Force kill a process (SIGKILL)
kill -9 1234

# Kill process by name
killall TextEdit

# Start a program
open -a TextEdit

# View process hierarchy
pstree
```

Environment Variables

Environment variables store system configuration:

- Windows:

```
# View all environment variables
set

# View specific variable
echo %PATH%
echo %USERPROFILE%

# Set a variable (temporary)
set MYVAR=Hello
echo %MYVAR%
```

- macOS:

```
# View all environment variables
env

# View specific variable
echo $PATH
echo $HOME

# Set a variable (temporary)
export MYVAR="Hello"
echo $MYVAR
```

Important Environment Variables

- `PATH`: **Directories where executable files are found** (separated by `:` in macOS, `;` in Win)
- `HOME`/`USERPROFILE`: User's home directory location
- `PWD`: Current working directory
- `TMPDIR`/`TEMP`: Directory for temporary files
- `USER`/`USERNAME`: Current username
- `SHELL`: Default shell program
- `LANG`/`LANGUAGE`: System language and locale settings

NOTE: The PATH environment variable is crucial for program accessibility. When software is installed, it must add its directory to PATH - otherwise you can only run the program by typing its absolute path. This is why properly installed programs can be launched from any location in the terminal by simply typing their name.

File Permissions

Windows Permissions:

- **Read**: View file contents
- **Write**: Modify file contents
- **Execute**: Run the file as a program
- **Full Control**: All permissions plus ability to change permissions

macOS Permissions (Unix-style):

```
# View permissions
ls -l myfile.txt
# Output: -rw-r--r-- 1 user group 1024 Jan 1 12:00 myfile.txt
#          ^^^ ^^^ ^^^
#          |   |   |
#          |   |   +-- Others (everyone else)
#          |   +------- Group
#          +----------- Owner (you)

# Change permissions
chmod 755 script.py    # rwxr-xr-x
chmod +x script.py     # Add execute permission
```

Permission Symbols:

- `r` = read (4), `w` = write (2), `x` = execute (1)

System Information Commands

- Windows:

```
# System information
systeminfo

# Current user
whoami

# Current date and time
date
time
```

- macOS:

```
# System information
uname -a

# Current user
whoami

# Current date and time
date

# Disk usage
df -h
```

Shortcuts and Productivity Tips

Windows Keyboard Shortcuts:

- `Win + E`: Open File Explorer
- `Win + R`: Run dialog
- `Win + L`: Lock computer
- `Alt + Tab`: Switch between applications
- `Ctrl + Shift + Esc`: Task Manager

macOS Keyboard Shortcuts:

- `Cmd + Space`: Spotlight search
- `Cmd + Tab`: Switch between applications
- `Cmd + Option + Esc`: Force quit applications
- `Cmd + Shift + 3`: Screenshot

Command Line Shortcuts (both systems):

- `Tab`: Auto-complete file/directory names
- `↑/↓ arrows`: Navigate command history
- `Ctrl + C`: Cancel current command
- `Ctrl + L`: Clear screen